Slovenská Technická Univerzita v Bratislave

Fakulta informatiky a informačných technológií

Komunikačný protokol pre IoT meteorologické zariadenia

Počítačové a komunikačné siete

Sophie Hetešová

AIS ID: 141435

Vedúci cvičenia: Bc. Filip Hromada
Čas cvičenia: Deň štvrtok – 8:00

Obsah

Úvod a architektúra riešenia **3**

Cieľ riešenia . . . 3

Rozdelenie programov . . . 3

Senzory a súbežné spracovanie . . . 3

Rozsah prototypu . . . 4

Návrh protokolu **4**

Prenos a identifikácia relácie . . . 4

Spoločné polia správ . . . 5

Typy správ a ich formát . . . 5

REGISTER – registrácia senzora . . . 5

ACK – potvrdenie registrácie alebo merania . . . 6

DATA – prenos merania . . . 6

RETRANSMIT – vyžiadanie správneho originálu . . . 7

REJECT – odmietnutie správy . . . 8

KEEPALIVE – overenie dostupnosti . . . 8

KEEPALIVE_ACK – odpoveď na overenie dostupnosti . . . 8

Validácia hodnôt merania . . . 9

Kontrola integrity CRC32 . . . 9

Časovanie a životný cyklus relácie . . . 10

Implementácia servera a testovacieho programu **12**

Dátové štruktúry a rozdelenie zodpovedností . . . 12

Prijímací a odosielací cyklus servera . . . 12

Prijímací a odosielací cyklus senzora . . . 15

Spustenie a ovládanie . . . 17

Validácia vstupov a spracovanie chýb . . . 17

Logovanie . . . 18

Použité knižnice 18

Sekvenčné diagramy a overenie UAT **20**

UAT 1 – registrácia a automatické merania 20

UAT 2 – dodatočné meranie používateľa 21

UAT 3 – poškodenie a retransmisia 22

UAT 4 – dostupnosť a opätovná registrácia 23

Automatické overenie a výsledky 24

Meranie efektivity protokolu **25**

Cieľ a opakovateľný scenár 25

Započítanie nižších vrstiev 25

Analytické veľkosti aktuálnych príkladov 26

Metriky a vyhodnotenie 26

Postup zachytávania a prezentácie výsledkov 27

Zhodnotenie riešenia **28**

Príloha A – prínos projektu **29**

Úvod a architektúra riešenia

Cieľ riešenia

Cieľom projektu je navrhnúť vlastný aplikačný protokol na prenos meteorologických meraní medzi simulovanými IoT senzormi a serverom. Komunikácia prebieha nad transportným protokolom UDP. Keďže UDP nezabezpečuje doručenie ani potvrdenie správ, registráciu senzorov, potvrdzovanie meraní a kontrolu dostupnosti rieši aplikačný protokol.

Správy sa prenášajú ako objekty JSON zakódované v UTF-8. Jeden UDP datagram obsahuje jednu správu. Čitateľný formát umožňuje kontrolovať obsah komunikácie pri ladení a pri sledovaní vo Wiresharku.

Rozdelenie programov

Riešenie je implementované v Pythone a pozostáva z troch súborov:

- `server.py` prijíma registračné požiadavky, vytvára relácie a prideľuje tokeny. Pri prijatí merania overuje reláciu, kontrolný súčet CRC32 a povolené hodnoty parametrov. Platné meranie potvrdzuje správou ACK, pri chybe integrity žiada jeho opätovné odoslanie. Dostupnosť senzorov sleduje pomocou správ KEEPALIVE.
- `tester.py` simuluje senzory, generuje ich merania a poskytuje konzolové menu. Používateľ môže odoslať vlastné hodnoty, poškodiť nasledujúcu dátovú správu, zapnúť alebo vypnúť automatické merania, ukončiť reláciu senzora a znova ho zaregistrovať.
- `lib.py` obsahuje spoločné funkcie na spracovanie JSON, výpočet a overenie CRC32, validáciu vstupov a logovanie. Uchováva aj rozsahy a presnosť jednotlivých meraných parametrov.

UDP komunikácia je v oboch programoch implementovaná priamo pomocou knižnice `socket`, vrátane vytvorenia a naviazania socketov. IP adresa a port servera sa zadávajú pri spustení. Na serveri sa zadaná adresa používa na počúvanie, v testeri ako cieľ komunikácie.

Senzory a súbežné spracovanie

Tester vytvára štyri simulované senzory:

- **ThermoNode** (`T-01`): teplota, relatívna vlhkosť, rosný bod a atmosférický tlak.

- **WindSense** (`W-01`): rýchlosť vetra, rýchlosť nárazu, smer vetra a index turbulencie.
- **RainDetect** (`R-01`): úhrn zrážok, vlhkosť pôdy, riziko povodne a trvanie zrážok.
- **AirQuality** (`A-01`): koncentrácia CO_2, koncentrácia ozónu a index kvality ovzdušia.

Každý senzor beží vo vlastnom vlákne a má samostatný UDP socket, frontu príkazov, token relácie, poradové čísla správ a časovače. Po úspešnej registrácii automaticky odošle prvé meranie a ďalšie generuje každých približne desať sekúnd. Čakanie jedného senzora na odpoveď neblokuje ostatné senzory. Hlavné vlákno testera obsluhuje menu a odovzdáva požiadavky príslušnému senzoru cez jeho frontu.

Server spracúva správy a časovače relácií v hlavnej sieťovej slučke. Samostatné vlákno číta príkazy jeho menu a odovzdáva ich cez frontu. Relácie tak spravuje iba sieťová slučka. Oba programy zapisujú udalosti a čitateľný obsah správ do vlastného logového súboru aj do konzoly. Pri novom spustení sa príslušný logový súbor prepíše.

Rozsah prototypu

Prototyp pokrýva registráciu a automatické merania (UAT 1), dodatočné meranie so zadanými hodnotami (UAT 2), detekciu poškodenia a retransmisiu na žiadosť servera (UAT 3) a kontrolu dostupnosti vrátane odpojenia a opätovnej registrácie (UAT 4). Tieto scenáre boli overené lokálnymi automatickými testami.

Návrh protokolu

Prenos a identifikácia relácie

Protokol používa UDP nad IPv4. Server počúva na adrese a porte zadaných používateľom. Každý senzor odosiela správy zo svojho UDP socketu s lokálnym portom prideleným operačným systémom. Server odpovedá na zdrojovú IP adresu a port prijatého datagramu. Správy nemajú samostatnú binárnu hlavičku; riadiace polia aj merania sú súčasťou jedného objektu JSON.

Senzor sa identifikuje neprázdnym reťazcom `sensor_id`, ktorý musí byť na serveri jedinečný aj medzi rôznymi typmi senzorov. Pri registrácii uvedie svoj typ v poli `sensor_type`. Podporované hodnoty sú `ThermoNode`, `WindSense`, `RainDetect` a `AirQuality`.

Server po úspešnej registrácii vytvorí náhodný token funkciou `secrets.token_hex(8)`. Token predstavuje osem náhodných bajtov zapísaných ako šestnásť hexadecimálnych znakov. Server si k ID uloží typ senzora, token a zdrojovú adresu s portom. Pri ďalšej dátovej správe overuje

všetky tieto údaje. Typ merania určuje z uloženej relácie, preto správa DATA neobsahuje pole `sensor_type`.

Token platí iba počas konkrétnej relácie. Po jej zrušení sa pôvodný token odmieta a nová úspešná registrácia vytvorí nový. Registračná požiadavka s už obsadeným ID sa odmietne, kým pôvodná relácia nezanikne.

Spoločné polia správ

Každá správa obsahuje polia `type` a `seq`. Ostatné polia závisia od jej typu.

Pole	Typ JSON	Význam
`type`	reťazec	Typ správy zapísaný veľkými písmenami.
`seq`	celé číslo	Kladné poradové číslo správy u jej odosielateľa.
`ack_for`	celé číslo	Poradové číslo správy, na ktorú odpoveď reaguje.
`sensor_id`	reťazec	Jedinečný identifikátor senzora.
`sensor_type`	reťazec	Typ senzora uvedený pri registrácii.
`token`	reťazec	Token pridelený serverom pre danú reláciu.
`values`	objekt	Hodnoty všetkých parametrov daného typu senzora.
`crc`	celé číslo	Nezáporný 32-bitový kontrolný súčet dátovej správy.
`reason`	reťazec	Čitateľný dôvod odmietnutia alebo žiadosti o retransmisiu.

Každý senzor má vlastný čítač `seq`, ktorý začína hodnotou 1. Server používa samostatný čítač pre svoje odoslané správy. Čísla rôznych senzorov sa preto môžu opakovať; odpoveď sa priraďuje cez socket konkrétneho senzora a pole `ack_for`. Nová správa dostáva nové číslo. Retransmisia zachováva číslo pôvodnej dátovej správy.

Typy správ a ich formát

Nasledujúce príklady používajú ukážkové tokeny. Odsadenie a zalomenia slúžia na čitateľnosť; program odosiela JSON bez nadbytočných medzier. Poradové čísla servera v príkladoch ilustrujú jednotlivé odpovede a nepredstavujú jeden súvislý záznam komunikácie.

REGISTER – registrácia senzora

Smer: senzor → server. Povinné polia sú `type`, `seq`, `sensor_type` a `sensor_id`. Token ešte neexistuje, preto sa neposiela. Server overí podporovaný typ a jedinečnosť ID.

```
{
  "type": "REGISTER", "seq": 1,
  "sensor_type": "ThermoNode", "sensor_id": "T-01"
}
```

ACK – potvrdenie registrácie alebo merania

Smer: server → senzor. Odpoveď obsahuje `type`, `seq` a `ack_for`. Pri úspešnej registrácii navyše obsahuje pridelený `token`:

```
{
  "type": "ACK", "seq": 1, "ack_for": 1,
  "token": "a1b2c3d4e5f60718"
}
```

Potvrdenie platného merania token neobsahuje. Nasledujúca odpoveď potvrdzuje spracovanie DATA s poradovým číslom 2:

```
{"type": "ACK", "seq": 2, "ack_for": 2}
```

DATA – prenos merania

Smer: senzor → server. Povinné polia sú `type`, `seq`, `sensor_id`, `token`, `values` a `crc`. Automatické aj ručne zadané meranie používajú rovnaký formát. Objekt `values` musí obsahovať presne všetky parametre registrovaného typu senzora.

ThermoNode:

```
{
  "type": "DATA", "seq": 2, "sensor_id": "T-01",
  "token": "a1b2c3d4e5f60718",
  "values": {
    "temperature": 21.3, "humidity": 55.4,
    "dew_point": 11.9, "pressure": 1013.25
  },
  "crc": 1819254021
}
```

WindSense:

```
{
  "type": "DATA", "seq": 2, "sensor_id": "W-01",
  "token": "b1c2d3e4f5061728",
  "values": {
    "wind_speed": 5.2, "wind_gust": 8.1,
    "wind_direction": 90, "turbulence": 0.3
  },
  "crc": 1840484505
}
```

RainDetect:

```
{
  "type": "DATA", "seq": 2, "sensor_id": "R-01",
  "token": "c1d2e3f405162738",
  "values": {
    "rainfall": 12.5, "soil_moisture": 60.0,
    "flood_risk": 1, "rain_duration": 20
  },
  "crc": 2586586403
}
```

AirQuality:

```
{
  "type": "DATA", "seq": 2, "sensor_id": "A-01",
  "token": "d1e2f30415263748",
  "values": {
    "co2": 800, "ozone": 45.2, "air_quality_index": 50
  },
  "crc": 2834147288
}
```

RETRANSMIT – vyžiadanie správneho originálu

Smer: server → senzor. Server odošle túto správu, ak relácia a zdrojová adresa sú platné, ale kontrola CRC zlyhá. Pole `ack_for` označuje poškodenú správu. Odpoveď obsahuje aj ID, token a dôvod žiadosti.

```
{
  "type": "RETRANSMIT", "seq": 3, "ack_for": 2,
  "sensor_id": "T-01", "token": "a1b2c3d4e5f60718",
  "reason": "Integrity check failed"
}
```

Tester overí ID a token, vyhľadá čakajúci originál podľa `ack_for` a odošle ho bez poškodenia. Zachová `seq`, hodnoty aj CRC. Pre jednu dátovú správu sú povolené najviac tri odoslania vrátane pôvodného.

REJECT – odmietnutie správy

Smer: server → senzor. Polia `type`, `seq`, `ack_for` a `reason` informujú o odmietnutí požiadavky. Príklad pre neplatný token:

```
{
  "type": "REJECT", "seq": 4, "ack_for": 2,
  "reason": "Invalid sensor ID or token"
}
```

Ďalšími dôvodmi sú už registrované ID, nesúlad zdrojovej adresy, neplatné parametre merania alebo opätovné použitie nedávneho `seq` pre odlišné dáta. Nečitateľný JSON, neznámy typ správy alebo neplatné `seq` sa zalogujú a zahodia bez protokolovej odpovede.

KEEPALIVE – overenie dostupnosti

Smer: server → senzor. Správa obsahuje `type`, `seq`, `sensor_id` a `token`. Server ju posiela po pätnástich sekundách bez platných dát a potom každých päť sekúnd, kým senzor zostáva bez meraní.

```
{
  "type": "KEEPALIVE", "seq": 5,
  "sensor_id": "T-01", "token": "a1b2c3d4e5f60718"
}
```

KEEPALIVE_ACK – odpoveď na overenie dostupnosti

Smer: senzor → server. Okrem ID a tokenu obsahuje `ack_for` s číslom prijatej sondy. Odpoveď používa vlastné nové `seq` senzora.

```
{
  "type": "KEEPALIVE_ACK", "seq": 3, "ack_for": 5,
  "sensor_id": "T-01", "token": "a1b2c3d4e5f60718"
}
```

Server akceptuje odpoveď iba pri zhode relácie, zdrojovej adresy a čísla poslednej čakajúcej sondy. Tým vynuluje počet nezodpovedaných sond. Čas posledného merania sa nemení.

Validácia hodnôt merania

Server aj používateľské rozhranie testera kontrolujú rozsahy a presnosť podľa nasledujúcej tabuľky. Obe hranice rozsahu sú povolené. Desatinné parametre môžu mať nižšiu presnosť, než je uvedené maximum; celočíselné parametre musia byť celé čísla JSON. Hodnoty `NaN` a nekonečno sa odmietajú.

Parameter	Rozsah	Presnosť
ThermoNode		
`temperature`	−50,0 až 60,0 °C	1 desatinné miesto
`humidity`	0,0 až 100,0 %	1 desatinné miesto
`dew_point`	−50,0 až 60,0 °C	1 desatinné miesto
`pressure`	800,00 až 1100,00 hPa	2 desatinné miesta
WindSense		
`wind_speed`	0,0 až 50,0 m/s	1 desatinné miesto
`wind_gust`	0,0 až 70,0 m/s	1 desatinné miesto
`wind_direction`	0 až 359 stupňov	celé číslo
`turbulence`	0,0 až 1,0	1 desatinné miesto
RainDetect		
`rainfall`	0,0 až 500,0 mm	1 desatinné miesto
`soil_moisture`	0,0 až 100,0 %	1 desatinné miesto
`flood_risk`	0 až 3	celé číslo
`rain_duration`	0 až 60 minút	celé číslo
AirQuality		
`co2`	300 až 5000 ppm	celé číslo
`ozone`	0,0 až 500,0 µg/m^3	1 desatinné miesto
`air_quality_index`	0 až 500	celé číslo

Pri `flood_risk` znamená 0 žiadne riziko, 1 nízke, 2 stredné a 3 vysoké riziko. Hodnoty v JSON používajú desatinnú bodku. Rozsah a presnosť sa kontrolujú aj na serveri nezávisle od používateľského vstupu.

Kontrola integrity CRC32

Kontrolný súčet obsahujú iba správy DATA. Vypočítava sa nad celým objektom správy bez poľa `crc`, teda aj nad ID, tokenom a poradovým číslom. Pred výpočtom sa objekt prevedie do jednotného tvaru nasledujúcim postupom:

1. Odstráni sa pole `crc`, ak je prítomné.

2. Kľúče sa zoradia abecedne aj vo vnorenom objekte `values`.
3. JSON sa zapíše bez medzier medzi položkami, s oddeľovačmi `,` a `:`. Ne-ASCII znaky sa zapíšu pomocou únikových sekvencií JSON. Nečíselné hodnoty `NaN` a nekonečno nie sú povolené.
4. Výsledný reťazec sa zakóduje do UTF-8 a vypočíta sa `zlib.crc32(data)`.
5. Výsledok v rozsahu 0 až 4 294 967 295 sa pridá do správy ako celé číslo `crc`.

Implementácia používa tento zápis:

```
data = json.dumps(
    content, sort_keys=True, separators=(",", ":"),
    ensure_ascii=True, allow_nan=False
).encode("utf-8")
crc = zlib.crc32(data)
```

CRC sa nepočíta priamo z pôvodných bajtov prijatého datagramu, ale z takto upraveného objektu. Zmena odsadenia alebo poradia kľúčov preto sama osebe nemení výsledok. Číselná reprezentácia musí zodpovedať použitej serializácii; napríklad JSON čísla `60` a `60.0` vedú v tejto implementácii k odlišnému vstupu pre CRC. Kontrolné súčty v uvedených príkladoch DATA boli vypočítané rovnakou funkciou ako v programe.

Pri simulácii chyby tester uchová originál a zmení jednu hodnotu iba v odosielanej kópii bez prepočítania CRC. Server poškodené meranie nepotvrdí ako platné a odošle RETRANSMIT. CRC slúži na odhalenie poškodenia, nie na kryptografické overenie pôvodu správy.

Časovanie a životný cyklus relácie

Po registrácii sa automatické merania zapnú. Senzor odošle prvé meranie ihneď a ďalšie približne každých 10 sekúnd. Ručné meranie nemení plán automatického odosielania. Na potvrdenie registrácie alebo DATA sa čaká najviac 3 sekundy. Bez odpovede sa čakajúca požiadavka zahodí; automatické časované opakovanie pri strate ACK nie je súčasťou tejto verzie. Opakovanie DATA prebieha na explicitnú žiadosť RETRANSMIT.

Ak server 15 sekúnd neprijme platné meranie, zaloguje `SENSOR IDLE` a odošle KEEPALIVE. Ak ešte žiadne meranie neprišlo, čas sa počíta od registrácie. Pri úplnom výpadku odošle sondy v časoch 15, 20 a 25 sekúnd a po ďalších piatich sekundách zruší reláciu s výpisom `SENSOR DISCONNECTED`. Vypnutie iba automatických meraní reláciu neruší, pretože senzor na sondy naďalej odpovedá. Nová platná DATA obnoví aktívny stav a pätnásťsekundový časovač.

Tester ukončí lokálnu reláciu aj vtedy, keď 30 sekúnd neprijme platný kontakt zo servera. Časovač obnovuje prijatie KEEPALIVE, ACK merania alebo platnej žiadosti RETRANSMIT. ACK sa započítava preto, že pri pravidelných meraniach server sondy KEEPALIVE neposiela. Všetky intervaly sa merajú pomocou `time.monotonic()`, aby ich neovplyvnila zmena systémového času.

Pri úplnom lokálnom odpojení senzor prestane posielať DATA aj odpovede na KEEPALIVE. Neposiela samostatnú správu o odpojení; server výpadok zistí časovačmi. Po zrušení serverovej relácie možno rovnaké ID znova zaregistrovať s novým tokenom.

Server si pre každú reláciu pamätá posledných 256 potvrdených poradových čísel spolu s CRC. Rovnakú správu v tomto okne znovu potvrdí, ale nezaeviduje ju ako nové meranie. Použitie rovnakého čísla pre obsah s odlišným CRC odmietne.

Implementácia servera a testovacieho programu

Dátové štruktúry a rozdelenie zodpovedností

Server uchováva relácie v slovníku `sessions`, ktorého kľúčom je ID senzora. Záznam obsahuje typ, token, adresu a port, čas poslednej DATA, stav nečinnosti, termín ďalšej sondy, počet nezodpovedaných sond a číslo čakajúcej KEEPALIVE. Samostatný slovník v relácii uchováva posledných 256 potvrdených poradových čísel a ich CRC. Funkcia `process_measurement()` kontroluje prijaté meranie a funkcia `monitor_sessions()` spravuje overovanie dostupnosti.

Tester vytvára štyri pracovné vlákna pomenované podľa ID senzorov. Každé má vlastný socket a vykonáva funkciu `run_sensor()`. Stav senzora obsahuje typ, ID, token, príznaky platnej relácie a automatiky, termín ďalšieho merania a čas posledného kontaktu so serverom. Čakajúce dátové správy sú uložené v `pending`, registračné požiadavky v `registrations`. Pri DATA sa uchováva správny originál, termín odpovede a počet odoslaní.

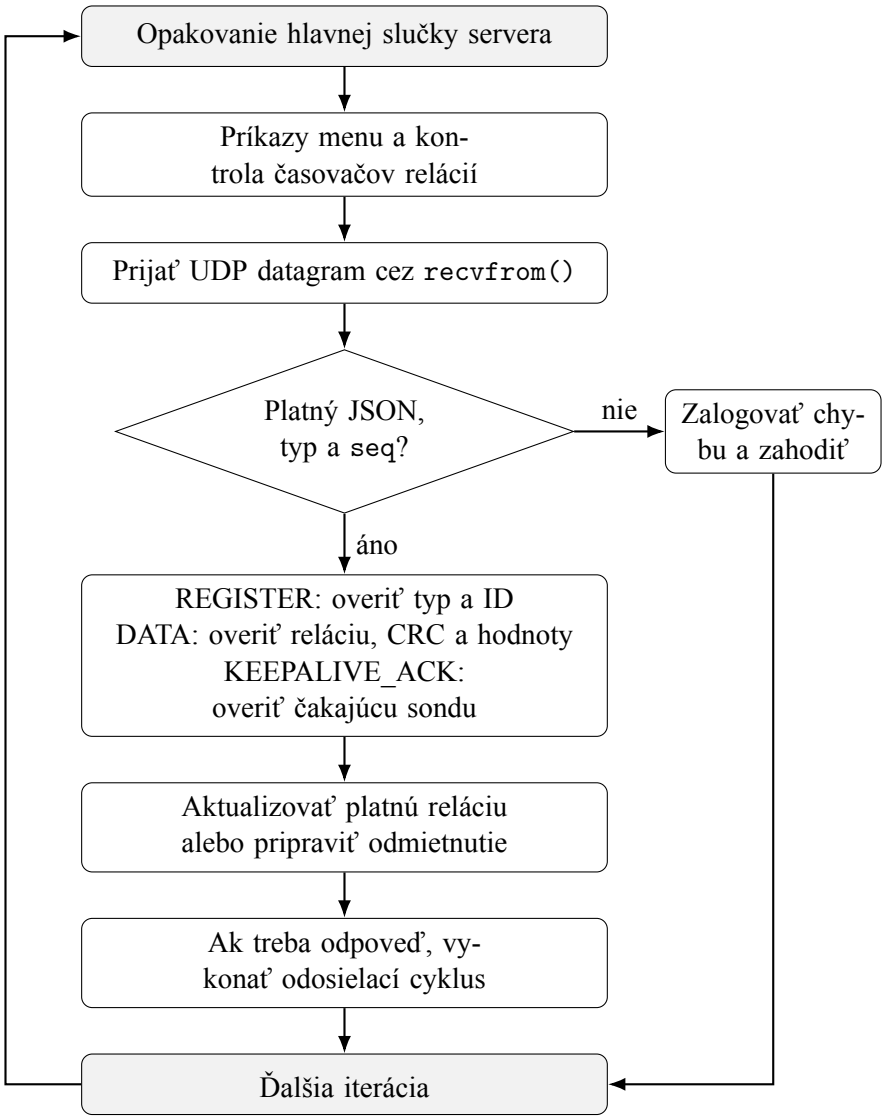
Menu nevykonáva sieťové operácie ani nemení tokeny. Požiadavku vloží do `queue.Queue` konkrétneho senzora. Jeho vlákno ju spracuje v ďalšej iterácii. Spoločný `threading.Event` oznamuje ukončenie celého testera; hlavné vlákno potom počká na skončenie všetkých senzorov pomocou `join()`. Sockety sa zatvoria opustením kontextu `with`. Lokálne odpojenie jedného senzora ukončuje iba jeho reláciu, vlákno zostáva pripravené na novú registráciu.

Prijímací a odosielací cyklus servera

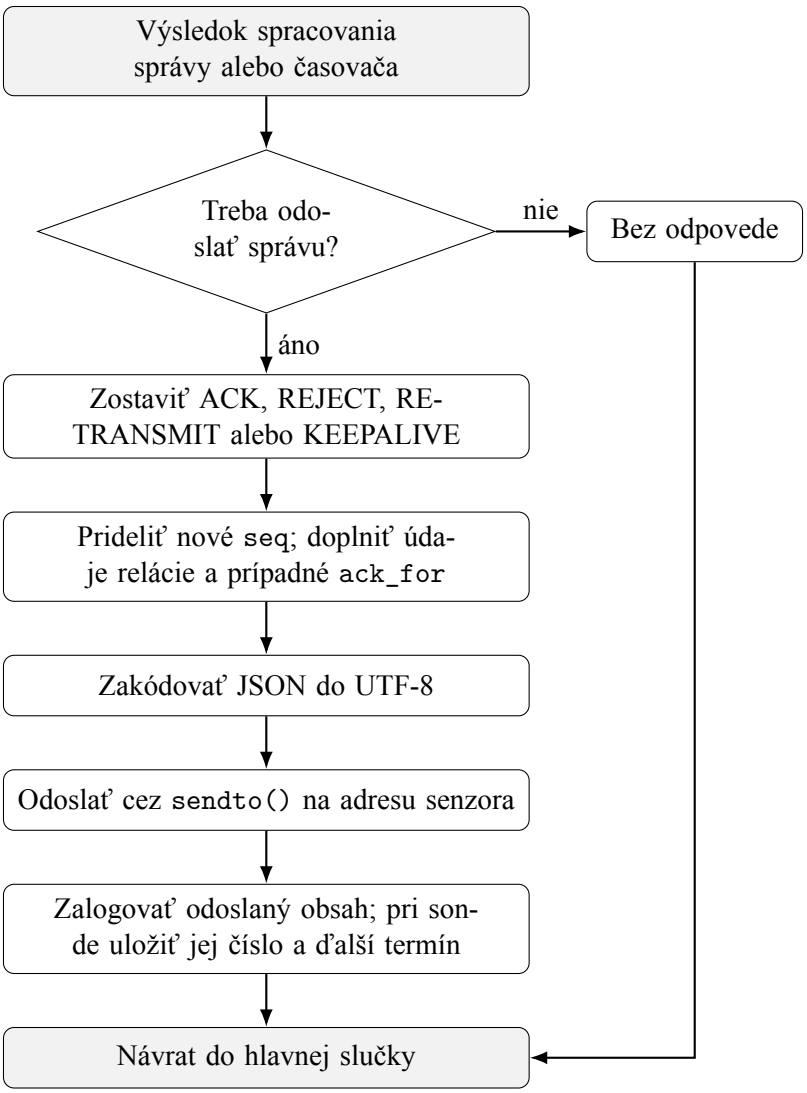
Server najprv spracuje príkazy menu a časovače relácií, potom čaká na datagram. Timeout socketu je najviac 0,2 sekundy, prípadne kratší podľa najbližšieho termínu sondy. Časovače preto fungujú aj vtedy, keď neprichádzajú žiadne správy. Prijímací cyklus na obrázku 1 rozlišuje registráciu, meranie a odpoveď na KEEPALIVE.

Pri DATA sa kontroluje relácia a zdrojová adresa, následne CRC a parametre merania. Platné nové meranie sa zaeviduje, zaloguje a potvrdí. Pri chybe integrity sa vyžiada originál; pri neplatnej relácii alebo hodnotách sa odošle odmietnutie. Platná odpoveď na KEEPALIVE nemení čas posledného merania, iba ruší stav nezodpovedanej sondy. Chyby dekódovania a neplatné riadiace polia sa zalogujú bez pádu prijímacej slučky.

Odosielanie servera je súčasťou tej istej sieťovej slučky, nie ďalšie vlákno. Vzniká ako reakcia na prijatú správu alebo na časovač. Obrázok 2 zobrazuje zostavenie odpovede, pridelenie poradového čísla, serializáciu a odoslanie. Pri KEEPALIVE sa navyše uloží číslo sondy a ďalší termín kontroly.



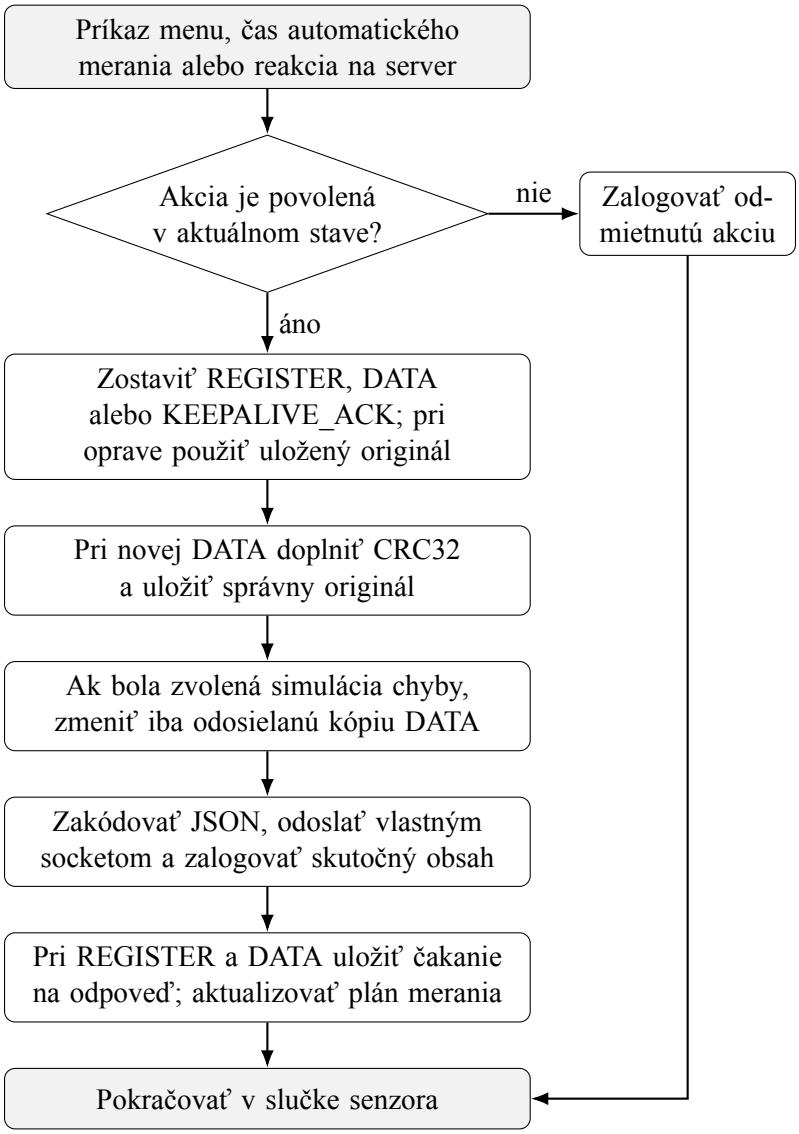
Obrázok 1: Prijímací cyklus servera a spracovanie stavu relácií.



Obrázok 2: Odosielací cyklus servera pre odpovede a sondy dostupnosti.

Prijímací a odosielací cyklus senzora

Každé vlákno senzora kombinuje odosielanie a prijímanie v jednej slučke. Najprv spracuje svoju frontu príkazov a vyhodnotí automatické meranie. Ak treba odoslať DATA, vytvorí hodnoty alebo použije používateľský vstup, pridá údaje relácie a vypočíta CRC. Správny originál si ponechá aj v prípade, že má byť odosielaná kópia poškodená. Postup je na obrázku 3.

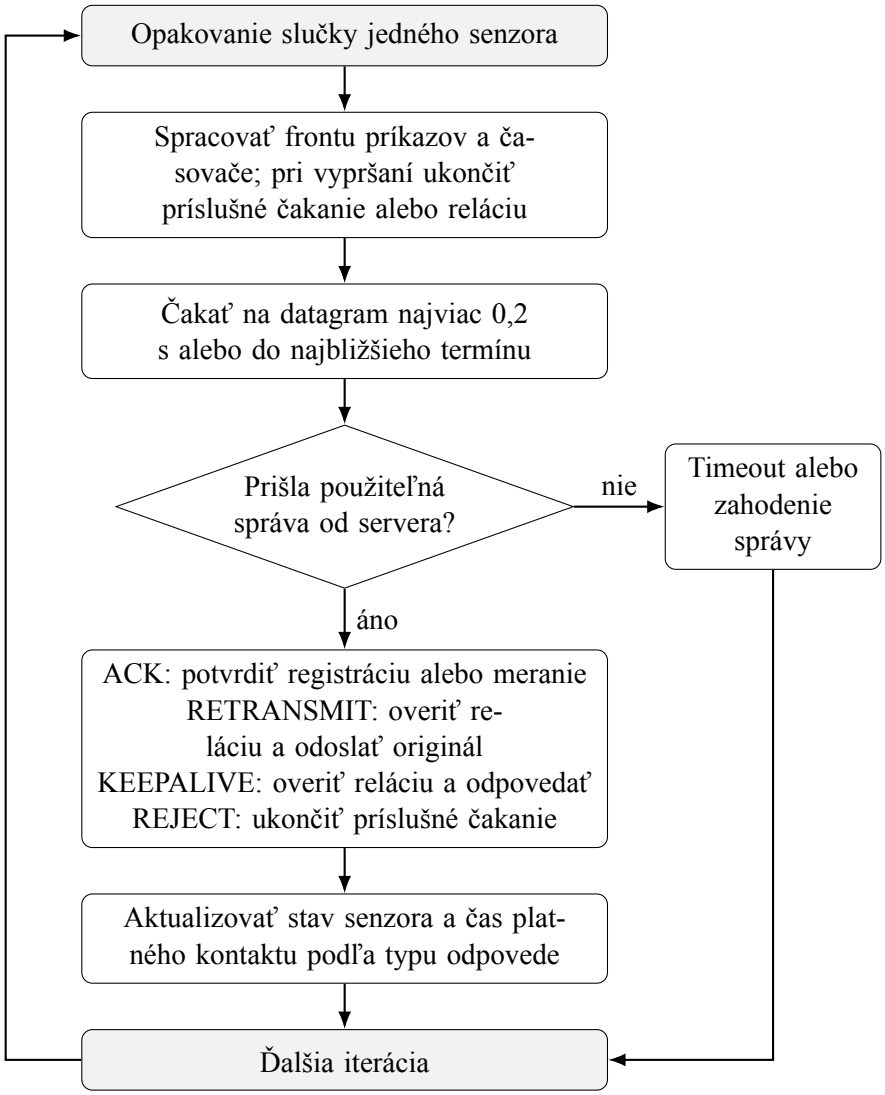


Obrázok 3: Odosielací cyklus jedného senzora testovacieho programu.

Prijímací cyklus na obrázku 4 overuje zdroj odpovede a dekóduje JSON. ACK sa priradí k čakajúcej požiadavke cez `ack_for`. Potvrdenie registrácie aktivuje reláciu a automatiku. RETRANSMIT vyvolá odoslanie uloženého originálu. Na KEEPALIVE s platným tokenom sa odpovedá aj pri vypnutých automatických meraniach. REJECT ukončí príslušné čakanie; odmietnutie tokenu navyše ukončí lokálnu reláciu.

Čakanie na socket trvá najviac 0,2 sekundy. Vlákno tak priebežne obsluhuje menu, časovače,

nové merania aj odpovede. Po troch sekundách bez odpovede odstráni čakajúcu požiadavku. Tým sa nezastaví plán ďalších automatických meraní.



Obrázok 4: Prijímací cyklus a spracovanie odpovedí v jednom vlákne senzora.

Spustenie a ovládanie

Programy sa spúšťajú z priečinka, ktorý obsahuje aj `lib.py`. Najprv sa spustí server a potom tester v druhom termináli:

```
python3 server.py
python3 tester.py
```

Oba programy si vypýtajú IPv4 adresu a UDP port. Pri lokálnom teste stačí potvrdiť predvolené hodnoty `127.0.0.1` a `9000`. Pri vzdialenej komunikácii tester používa skutočnú adresu servera. Server môže počúvať na `0.0.0.0`, teda na všetkých lokálnych IPv4 rozhraniach. Zdrojový port každého senzora prideľuje operačný systém.

Voľba	Operácia v menu testera
0	Ukončiť celý tester a všetky vlákna senzorov.
1	Vybrať senzor, zadať a odoslať vlastné meranie.
2	Označiť senzor na poškodenie jeho nasledujúcej DATA.
3	Vypnúť alebo zapnúť automatické merania vybraného senzora.
4	Ukončiť lokálnu reláciu vybraného senzora bez odpovedí na sondy.
5	Znova zaregistrovať odpojený senzor.

Menu servera obsahuje voľbu 0 na ukončenie. Oba programy možno ukončiť aj cez Ctrl+C. Po reštarte servera sa pôvodné tokeny nepoužívajú, preto je potrebná nová registrácia senzorov.

Validácia vstupov a spracovanie chýb

IP adresu kontroluje `ipaddress.IPv4Address`, port musí byť celé číslo v rozsahu 1 až 65 535. Neplatný vstup sa zaloguje a používateľ ho môže opraviť. Chyba naviazania socketu, napríklad obsadený port, sa zachytí ako `OSError` a zapíše do logu. Program neprejde do stavu počúvania, ak naviazanie zlyhalo.

Ručné meranie sa kontroluje ešte pred odoslaním. Desatinný vstup sa najprv spracuje pomocou `Decimal`, aby prevod na `float` neskryl nepovolenú presnosť. Vstup s nepovolenými desatinnými miestami sa odmietne aj vtedy, keď by sa zaokrúhlil na platnú hodnotu. Nepovolená hodnota sa zaloguje a rovnaký parameter sa zadáva znova. Server rozsahy, dátové typy a úplnosť `values` overuje nezávisle.

Predčasná opätovná registrácia neprevezme existujúcu reláciu. Server odpovie REJECT a ostatné senzory pokračujú. Neplatné JSON správy ani cudzie odpovede nesmú byť považované za potvrdenie merania. Očakávané chyby vstupu a komunikácie sa spracúvajú s čitateľným logom.

Logovanie

Každý program používa vlastný súbor: `server.log` alebo `tester.log`. Log sa otvorí v režime prepisovania pri spustení, takže obsahuje iba aktuálny beh vrátane chýb pri konfigurácii. Výpis do konzoly je v aktuálnom kóde zapnutý. Príslušná funkcia podporuje jeho vypnutie parametrom `console`, menu tento prepínač neposkytuje.

Súborové záznamy majú anglický obsah a jednotnú štruktúru:

```
<TIMESTAMP> | <TYPE> | <LEVEL> | <SOURCE> | <MESSAGE>
```

Čas je v UTC vo formáte `YYYY-MM-DDTHH:mm:ss.sssZ`. SYSTEM označuje lokálnu udalosť a MESSAGE prijatie alebo odoslanie správy. Použité závažnosti sú INFO, WARNING a ERROR. Zdrojom je SERVER, TESTER alebo konkrétny senzor, napríklad `TESTER:ThermoNode:T-01`. Každá prijatá a odoslaná protokolová správa sa zaznamenáva v čitateľnom tvare vrátane hodnôt merania. Pri simulovanej chybe odosielací log obsahuje skutočnú poškodenú kópiu, nie iba uložený originál.

Konzola používa samostatný formátovač: zobrazuje skrátený čas v UTC, zdroj, smer TX alebo RX, typ správy a prehľadné hodnoty s jednotkami. V interaktívnom termináli sú senzory a závažnosti farebne odlíšené. Pri presmerovaní výstupu sa farby nepoužijú; možno ich vypnúť aj premennou prostredia `NO_COLOR`. Výpisy z pracovných vlákien sa zapisujú pod spoločným zámkom a pri zadávaní sa obnovuje prompt aj rozpracovaný vstup. Obsah protokolových správ v súborovom logu sa tým nemení.

Použité knižnice

Aplikácia používa iba štandardnú knižnicu Pythonu. Nie je potrebná inštalácia externých komunikačných balíkov.

Modul	Použitie
`socket`	Vytvorenie, naviazanie a obsluha UDP socketov.
`threading`	Vlákna senzorov, menu servera a signál na ukončenie.
`queue`	Bezpečné odovzdávanie príkazov medzi vláknami.
`json`	Serializácia a dekódovanie protokolových správ.
`zlib`	Výpočet CRC32.
`secrets`	Generovanie tokenov relácií.
`random`	Generovanie simulovaných meraní.
`copy`	Kópia DATA určená na simulované poškodenie.
`ipaddress`	Validácia IPv4 adries.
`decimal`	Presná kontrola číselného vstupu a desatinných miest.
`time`	Monotónne časovače a UTC čas pre logovanie.
`logging`	Zápis logov zo všetkých vlákien do súboru a konzoly.
`os`, `sys`	Nastavenie farieb a práca s výstupným prúdom terminálu.
`shutil`	Zistenie šírky terminálu pre prehľadné zalamovanie.
`readline`	Obnovenie rozpracovaného vstupu, ak je modul dostupný.

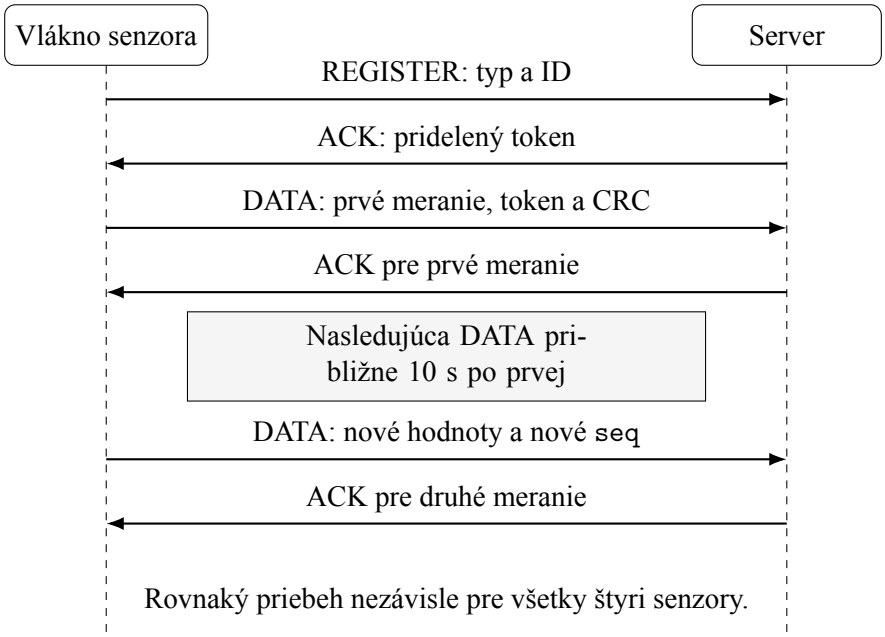
Na riadenie testov slúžia moduly `unittest` a `unittest.mock`. Procesy a dočasné súbory obsluhujú `subprocess`, `tempfile` a `pathlib`. Ďalej sa používajú `datetime`, `signal` a pomocné moduly `io`, `os` a `sys`. Testy spúšťajú aplikácie v dočasnom priečinku, aby neprepisovali používateľské logy.

Sekvenčné diagramy a overenie UAT

Diagramy zachytávajú komunikáciu jedného senzora so serverom. Ostatné senzory vykonávajú rovnaké operácie nezávisle vo vlastných vláknach. Čas v diagramoch postupuje zhora nadol.

UAT 1 – registrácia a automatické merania

Server počúva na zvolenej adrese a porte. Tester spustí štyri senzory a každý odošle REGISTER. Server pridelí vlastný token každej relácii. Po prijatí registračného ACK senzor odošle prvú DATA a pokračuje v desaťsekundových intervaloch. Obrázok 5 zobrazuje registráciu a dve merania.

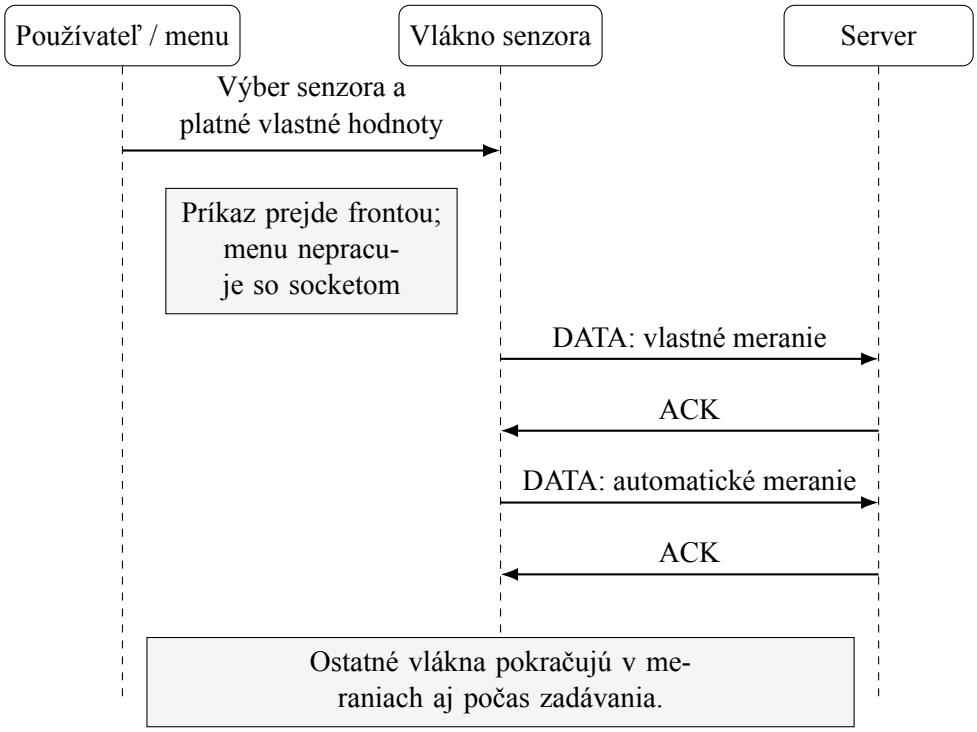


Obrázok 5: Sekvenčný diagram UAT 1 pre jeden zo štyroch senzorov.

Pri overení sa kontrolujú štyri úspešné registrácie a aspoň dve potvrdené DATA každého senzora. Súčasťou hraničných skúšok je zadanie chybnej IP adresy a neplatného portu. Program musí odmietnuť vstup bez pádu, umožniť opravu a zapísať chybu do logu.

UAT 2 – dodatočné meranie používateľa

Používateľ zvolí voľbu 1, vyberie registrovaný senzor a zadá hodnoty všetkých jeho parametrov. Tester odošle dodatočnú DATA so samostatným poradovým číslom. Server zaloguje presne tieto hodnoty a pošle ACK. Ručný vstup nemení plán automatických meraní ani komunikáciu ostatných senzorov, ako znázorňuje obrázok 6.

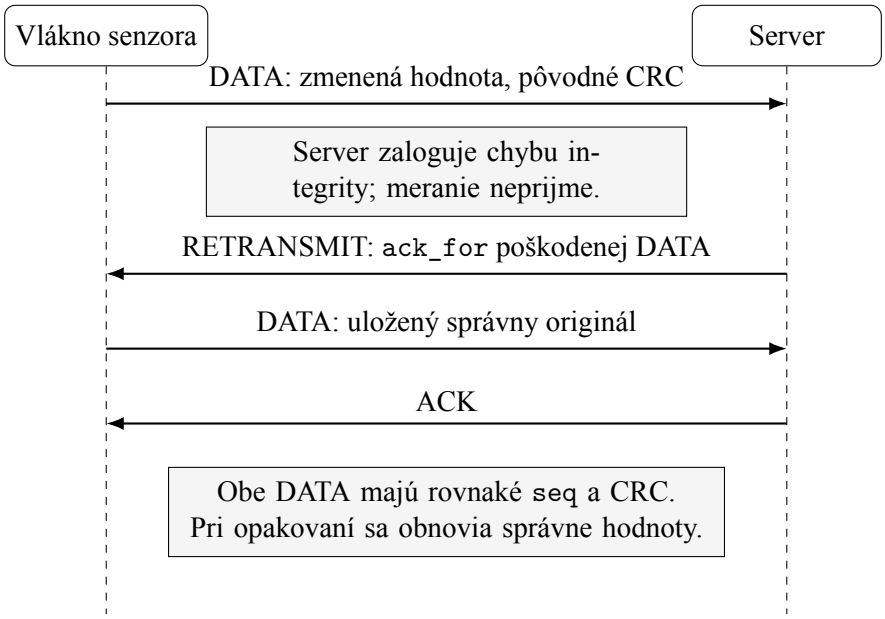


Obrázok 6: Sekvenčný diagram UAT 2 s oddelením menu a vlákna senzora.

Pri hraničnej skúške sa pre ThermoNode zadá napríklad teplota 61. Program ju odmietne, zaloguje dôvod a znova vyžiada teplotu. Po oprave na hodnoty 21.3, 55.4, 11.9 a 1013.25 sa overí ich presný príjem serverom. Neplatné rozpracované meranie sa do siete neposiela.

UAT 3 – poškodenie a retransmisia

Voľba 2 označí nasledujúcu DATA vybraného senzora na poškodenie. Tester zmení jednu hodnotu v kópii až po výpočte CRC a bez osobitného oznámenia serveru. Server odmietne poškodený obsah a pošle RETRANSMIT. Tester odošle uložený správny originál. Až tento originál sa prijme ako meranie a potvrdí, ako zobrazuje obrázok 7.

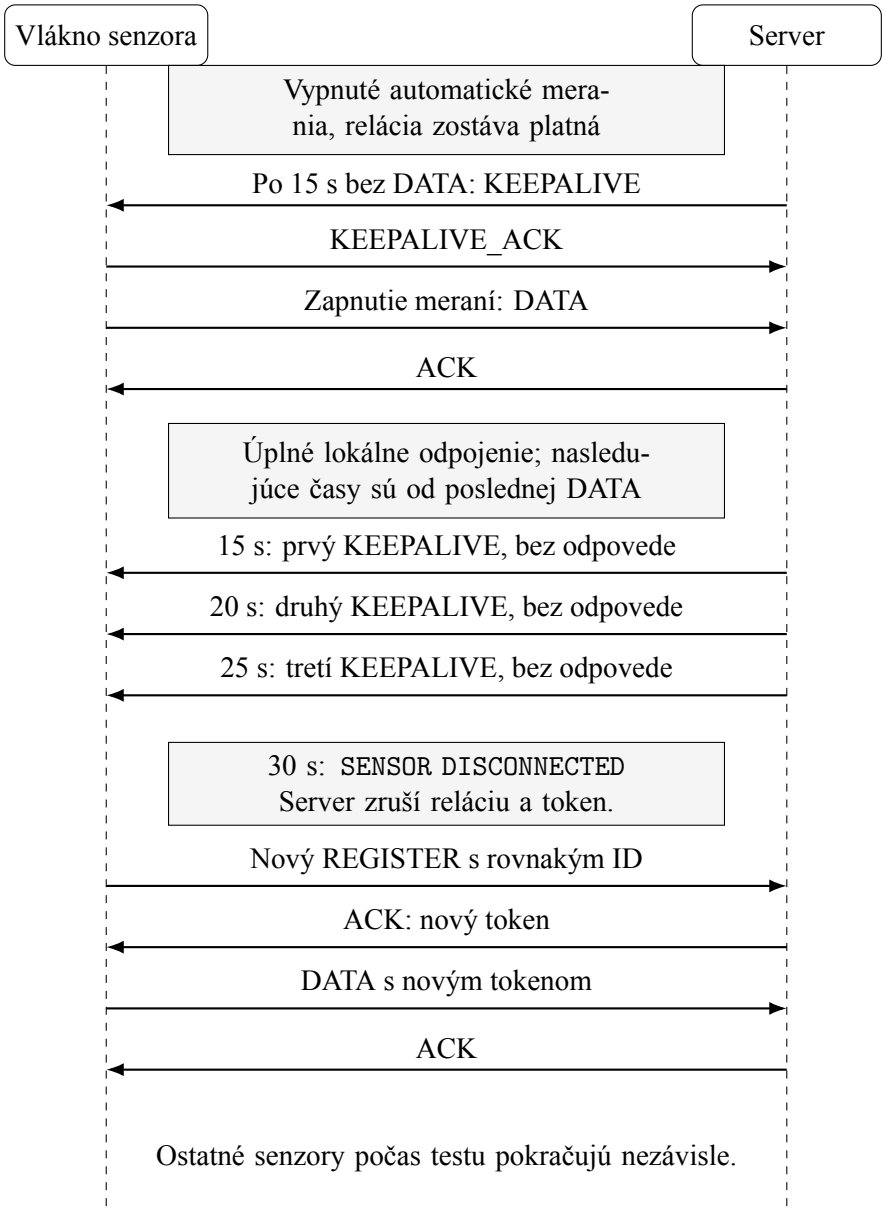


Obrázok 7: Sekvenčný diagram UAT 3: odhalenie poškodenia a oprava merania.

Kontroluje sa zhoda `seq` a `crc` oboch odoslaní, rozdiel poškodenej hodnoty a následné potvrdenie správneho originálu. Scenár bol overený pre všetky štyri typy senzorov. Zapnutie poškodenia pri vypnutej automatike sa uplatní na najbližšie ručné alebo neskôr obnovené automatické meranie.

UAT 4 – dostupnosť a opätovná registrácia

Voľba 3 vypne iba automatické merania. Po pätnástich sekundách server vyhlási senzor za nečinný, ale prijíma jeho odpovede na KEEPALIVE a reláciu ponechá platnú. Po opätovnom zapnutí sa musí prijať aspoň jedno meranie. Následná voľba 4 ukončí lokálnu reláciu vrátane odpovedí na sondy. Server po troch nezodpovedaných sondách a poslednej päťsekundovej lehote zruší token. Voľba 5 potom vytvorí novú reláciu. Celý priebeh je na obrázku 8.



Obrázok 8: Sekvenčný diagram UAT 4: nečinnosť, úplné odpojenie a nový token.

Predčasná registrácia počas stále platnej starej relácie sa odmieta. Po jej zrušení musí nový token umožniť prenos a pôvodný token sa nesmie akceptovať. Správanie časovačov nie je naviazané iba na menu: bez dát a odpovedí rovnakým spôsobom zanikne aj relácia senzora, ktorého sieťové spojenie sa preruší.

Automatické overenie a výsledky

Súbor `tests/test_presentation.py` obsahuje kontroly vstupov a lokálne integračné testy. Bežné spustenie vykoná kontroly bez sieťových socketov. Nastavenie premennej prostredia zapne aj testy s reálnymi UDP socketmi a samostatnými procesmi servera a testera:

```
python3 -m unittest discover -s tests -v
PKS_RUN_UDP_TESTS=1 python3 -m unittest discover -s tests -v
```

Pri poslednom vykonaní celej sady po rozdelení senzorov do samostatných vlákien prešlo všetkých 10 testov. Celkový čas bol približne 60,5 sekundy; jeho významnú časť tvorí čakanie na skutočné intervaly KEEPALIVE. Pri lokálnom scenári bolo zrušenie relácie zaznamenané 30,001 sekundy od posledného platného merania. Nejde o meranie výkonu protokolu, ale o kontrolu časového správania konkrétneho behu.

Kontrolovaná situácia	**Lokálny výsledok**
Neplatná IP a port, oprava a logovanie	Úspešné
Hranice rozsahov všetkých parametrov	Úspešné
Odmietnutie neplatnej presnosti pred prevodom na float	Úspešné
Ručné hodnoty mimo rozsahu a následné platné meranie	Úspešné
Registrácia štyroch senzorov s rôznymi UDP portmi	Úspešné
Poškodenie a oprava DATA každého typu	Úspešné
KEEPALIVE pri vypnutých meraniach	Úspešné
Odpojenie, zrušenie relácie a nový token	Úspešné
Neodpovedajúci senzor neblokuje ostatné tri	Úspešné
Ukončenie všetkých štyroch pracovných vlákien	Úspešné

Integračné testy používajú lokálnu adresu 127.0.0.1 a dočasné logy. Tabuľka sumarizuje overené situácie, nie samostatné body akceptačného hodnotenia. Neobsahuje výsledky zachytávania na vzdialenom sieťovom rozhraní.

Meranie efektivity protokolu

Cieľ a opakovateľný scenár

Cieľom merania je porovnať objem komunikácie aktuálneho JSON formátu s jeho budúcou binárnou reprezentáciou. V tejto dokumentácii je pripravený postup a analytický výpočet veľkostí existujúcich správ. Výsledky sieťového porovnania s binárnou verziou zatiaľ nie sú k dispozícii.

Základný scenár bude používať rovnaké štyri senzory a rovnakú sadu vopred uložených meraní pre oba formáty. Pre každý senzor sa odošle 60 meraní s intervalom 10 sekúnd, spolu 240 DATA. Meranie zahŕňa registráciu aj všetky odpovede. Pri úspešnom prenose bez opráv je to 4 REGISTER, 4 registračné ACK, 240 DATA a 240 dátových ACK, spolu 488 aplikačných datagramov. Počet meraní je určený explicitne, aby do výsledku náhodne nepribudlo meranie na konci časového okna.

Samostatné scenáre zachytia opravu jednej poškodenej DATA a nečinnosť jedného senzora s KEEPALIVE. Nemiešajú sa so základným prenosom. Pri jednej úspešnej oprave oproti základnému scenáru pribudne RETRANSMIT a jedno opakovanie DATA. Pre oba formáty sa zachovajú rovnaké hodnoty, počty správ, intervaly, dĺžky identifikátorov a podmienky siete. Binárna verzia musí zabezpečovať tie isté komunikačné funkcie.

Započítanie nižších vrstiev

Veľkosť aplikačnej správy sa určuje ako počet odoslaných bajtov, nie počet znakov vo formátovanom výpise. Pre JSON ide o `len(encode_message(message))`. Úroveň L3 navyše zahŕňa hlavičky UDP a IPv4. Pri IPv4 bez voliteľných polí a bez fragmentácie platí:

$$L_{\mathrm{L3}} = L_{\mathrm{app}} + 8 + 20.$$

Pre Ethernet II bez VLAN, s FCS a minimálnou dĺžkou rámca 64 bajtov, sa použije model:

$$L_{\mathrm{L2}} = \max\bigl(64,\ 14 + 20 + 8 + L_{\mathrm{app}} + 4\bigr).$$

Započítava sa tak aj prípadná výplň minimálneho ethernetového rámca. Ak sa samostatne vyhodnotí obsadenie média, k rámcu sa pripočíta 8 bajtových časov pre preambulu a začiatok rámca a 12 pre medzirámcovú medzeru. Tieto položky sa nebudú označovať za zachytené aplikačné dáta.

Pri zachytávaní treba uviesť typ rozhrania a overiť, či záznam obsahuje FCS. Ak ho adaptér

odstraňuje, analyticky doplnené štyri bajty sa odlíšia od zachytenej dĺžky. Loopback ani Wi-Fi sa nebudú vydávať za Ethernet II; pre iný typ linky sa použije jeho skutočná réžia. Rovnako sa oddelí prípadná VLAN a fragmentácia. Do porovnania oboch formátov sa zahrnú rovnaké vrstvy a rovnaký model.

Analytické veľkosti aktuálnych príkladov

Nasledujúce hodnoty boli vypočítané serializáciou príkladov DATA z kapitoly o protokole pomocou aktuálnej funkcie `encode_message()`. Ide o konkrétne správy so `seq=2`, uvedeným tokenom a CRC. Nie sú to priemery náhodne generovaných meraní ani výsledky Wiresharku.

Správa	JSON [B]	IPv4 + UDP [B]	Ethernet [B]
ThermoNode DATA	168	196	214
WindSense DATA	167	195	213
RainDetect DATA	168	196	214
AirQuality DATA	143	171	189
Dátové ACK	34	62	80

ACK v tabuľke má obsah `{"type":"ACK","seq":2,"ack_for":2}`. Jedno potvrdené meranie preto v tomto ethernetovom modeli zaberá napríklad 294 bajtov pre ThermoNode: 214 bajtov DATA a 80 bajtov ACK. Skutočný JSON objem sa mení s počtom číslic hodnôt, poradových čísel aj CRC. Pre výsledné porovnanie sa preto bude sčítavať celý rovnaký komunikačný scenár.

Metriky a vyhodnotenie

Pre každý formát sa zaznamená počet datagramov, súčet aplikačných bajtov, súčet bajtov na L3 a L2 a počet opravných alebo riadiacich správ. Úspora binárneho formátu sa vyjadrí pre zvolenú rovnakú úroveň vzorcom:

$$U = 100 \left(1 - \frac{B_{\text{binary}}}{B_{\text{JSON}}} \right) \%.$$

Pri rozlišovaní meraní a réžie sa použije rovnaký referenčný obsah na oboch stranách. Minimálny počet bitov na hodnotu s daným počtom prípustných stavov N je $\lceil \log_2 N \rceil$. Po prevode desatinných hodnôt na celé indexy podľa rozsahu vychádzajú tieto dolné hranice samotných parametrov:

Senzor	Bity jednotlivých parametrov	Spolu
ThermoNode	$11 + 10 + 11 + 15$	47 bitov
WindSense	$9 + 10 + 9 + 4$	32 bitov
RainDetect	$13 + 10 + 2 + 6$	31 bitov
AirQuality	$13 + 13 + 9$	35 bitov

Tieto počty nepredstavujú hotovú binárnu správu: nezahŕňajú identifikáciu, token, poradové čísla, CRC ani sieťové hlavičky. Umožňujú však uviesť rovnakú referenčnú informačnú hodnotu merania bez zvýhodnenia jedného formátu. Referenčný podiel meraní v celom prenose možno vyjadriť ako súčet týchto bitov delený celkovým počtom prenesených bitov na vybranej vrstve. Nejde o presné rozdelenie textového JSON na fyzické polia hlavičky a payloadu.

Postup zachytávania a prezentácie výsledkov

1. Uložiť vstupné merania a nastavenie scenára, vrátane verzie programov, typu rozhrania a počtu správ.
2. Spustiť zachytávanie pred prvou registráciou a filtrovať komunikáciu podľa UDP portu servera, aby sa zahrnuli všetky štyri lokálne porty senzorov.
3. Vykonať presne stanovený scenár a po poslednom potvrdení záznam ukončiť.
4. Oddeliť aplikačnú komunikáciu od inej prevádzky siete. Z exportu a logov overiť počty REGISTER, DATA, ACK a prípadných opráv alebo sond.
5. Rovnaký postup zopakovať s binárnym formátom, rovnakými hodnotami a rovnakým modelom započítania nižších vrstiev.
6. Výsledky uviesť v tabuľke a stĺpcovom grafe celkových bajtov. Samostatne zobraziť základný prenos a scenáre s opravou či nečinnosťou.

Základný aj doplnkové scenáre sa vykonajú aspoň trikrát. Uvedie sa prípadný rozdiel v počte správ spôsobený stratami alebo časovaním. Dĺžka záznamu, počet meraní a započítané vrstvy musia byť pri oboch formátoch porovnateľné. Zníženie počtu bajtov samo osebe nepredstavuje odmeranú úsporu energie batérie.

Zhodnotenie riešenia

Riešenie poskytuje vlastnú aplikačnú komunikáciu nad UDP s registráciou relácií, potvrdzovaním meraní, kontrolou integrity a kontrolou dostupnosti. Samostatné vlákna a sockety senzorov umožňujú súbežný prenos aj počas používateľského vstupu alebo neúspešnej registrácie jedného zariadenia. Čitateľný JSON a spoločný formát logov umožňujú sledovať jednotlivé kroky bez potreby špeciálneho dekodéra.

Lokálne testy potvrdili požadované správanie scenárov UAT 1 až 4 vrátane neplatných vstupov, opravy poškodenej správy a novej registrácie po odpojení. Nezávislosť vlákien bola overená aj prípadom, keď jeden senzor nedostal registračné potvrdenie a zvyšné tri ďalej odosielali merania.

Aktuálna verzia má aj obmedzenia. Tester vytvára pevne nastavené štyri senzory a menu zatiaľ nepridáva ďalšie inštancie rovnakého typu. Výpis do konzoly sa nastavuje v kóde. Pri strate ACK sa správa automaticky neopakuje; retransmisia je implementovaná na žiadosť servera pri chybe integrity. Token označuje reláciu a CRC32 odhaľuje poškodenie, ale komunikácia neposkytuje šifrovanie ani kryptografické overenie pôvodu.

Lokálny časovač senzora sa obnovuje aj platnými potvrdeniami meraní. Je to zvolená interpretácia nejednoznačnosti požiadavky na čas od poslednej KEEPALIVE, keďže pri pravidelných dátach server tieto sondy neposiela. Túto vlastnosť treba pri posudzovaní časovania zohľadniť.

Ďalšie optimalizácie môžu zmenšiť objem nahradením názvov polí kompaktnými kódmi a uložením parametrov podľa ich rozsahu a presnosti. Reálny rozdiel sa má vyhodnotiť podľa pripraveného meracieho postupu, vrátane riadiacich správ a nižších vrstiev. Zatiaľ dostupné analytické veľkosti JSON nie sú výsledkom porovnania dvoch implementovaných formátov.

Príloha A – prínos projektu

Pri práci na projekte som si precvičila návrh aplikačného protokolu a pochopila rozdiel medzi transportnou službou UDP a pravidlami, ktoré musí zabezpečiť samotná aplikácia. Naučila som sa pracovať so socketmi, identifikovať relácie pomocou tokenov a priraďovať odpovede k správam cez poradové čísla. Kontrola CRC mi ukázala, ako rozpoznať poškodenie dát a prečo je potrebné uchovať správny originál pre retransmisiu.

Rozdelenie senzorov do samostatných vlákien mi pomohlo pochopiť súbežné spracovanie, odovzdávanie príkazov cez fronty a bezpečné ukončenie programu. Pri časovačoch som si uvedomila význam monotónneho času a presného rozlíšenia medzi nečinnosťou senzora a úplným výpadkom spojenia. Testovanie neplatných vstupov ma viedlo k dôslednejšej validácii a k používaniu logov pri hľadaní chýb.

Tieto skúsenosti môžem využiť pri ďalších sieťových zadaniach, distribuovaných aplikáciách aj pri práci s IoT zariadeniami. Návrh opakovateľných testov a merania efektivity mi zároveň pomáha posudzovať riešenie podľa overiteľných výsledkov, nielen podľa toho, či sa program spustí. V praxi sú užitočné aj pri tímovom vývoji a vysvetľovaní technických rozhodnutí.